

Dokumentacja techniczna

Projekt System Aukcyjny

Zespół projektowy:

Adrian Dej, 67756

Bartosz Dej, 67757

Sebastian Serafin, 67486

Piotr Szych, 67374

Repozytorium: FiguSSo/Projekt-System-Aukcyjny

<https://github.com/FiguSSo/Projekt-System-Aukcyjny>

Technologia: Java 17, Spring Boot, JPA/Hibernate, Thymeleaf, SQLite

Dokumentacja projektu "System Aukcyjny" wykonanego w ramach przedmiotu Tworzenie usług sieciowych REST.

Spis treści

Spis treści można odświeżyć po otwarciu dokumentu w Microsoft Word lub LibreOffice.

1. Informacje ogólne
2. Architektura systemu
3. Stos technologiczny
4. Model danych
5. Warstwa REST API
6. Interfejs użytkownika
7. Walidacja i obsługa błędów
8. Trwałość danych i konfiguracja
9. Instrukcja uruchomienia

1. Informacje ogólne

Projekt System Aukcyjny jest aplikacją webową zbudowaną w architekturze warstwowej. System pozwala na rejestrację użytkowników, dodawanie aukcji, składanie ofert oraz przeglądanie aktualnie najwyższych ofert. Część funkcjonalności udostępniono jako REST API, a część jako interfejs HTML renderowany po stronie serwera.

2. Architektura systemu

Projekt został wykonany w architekturze warstwowej.

- Controller – obsługa żądań HTTP
- Service – logika biznesowa
- Repository – dostęp do bazy danych
- Model – encje aplikacji
- DTO – obiekty przesyłane przez API

Przepływ danych:

Klient → Controller → Service → Repository → Baza danych

3. Stos technologiczny

W projekcie wykorzystano:

- Java 17
- Spring Boot
- Spring Data JPA
- Hibernate
- SQLite
- Thymeleaf
- Swagger/OpenAPI

4. Model danych

System operuje na trzech głównych encjach: User, Auction oraz Bid. Relacje między nimi stanowią rdzeń modelu domenowego.

Encja	Najważniejsze pola	Opis
User	id, username, password, email, firstName, lastName	Użytkownik systemu. Dane są zapisywane w tabeli users.
Auction	id, title, description, startingPrice, startDate, endDate, category, status,	Aukcja zawierająca podstawowe informacje o przedmiocie oraz referencję do aktualnie

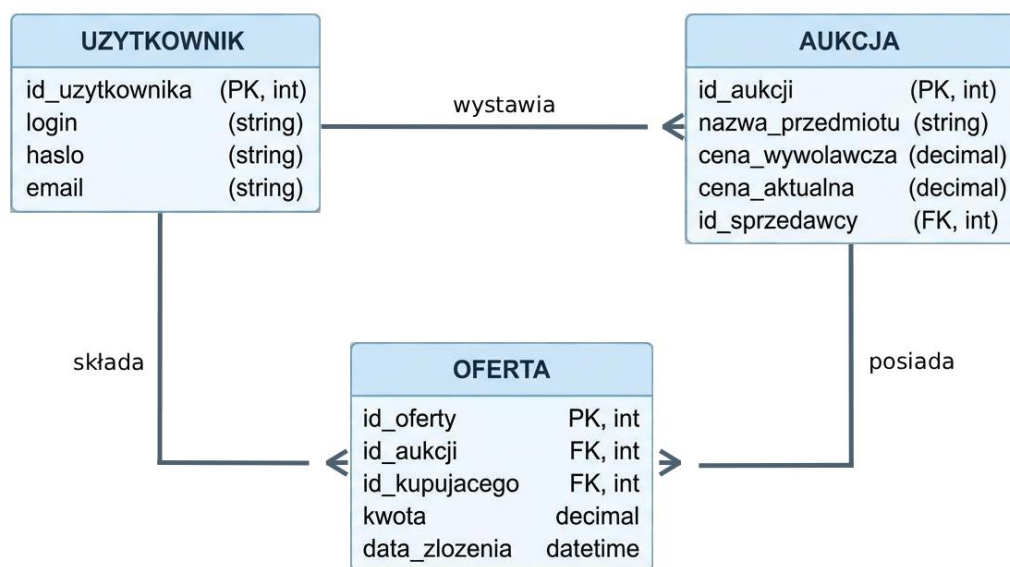
	currentHighestBid	najwyższej oferty.
Bid	id, amount, bidTime, auction, bidder	Pojedyncza oferta przypisana do konkretnej aukcji i użytkownika.

Relacje w modelu:

- Bid -> Auction: wiele ofert może dotyczyć jednej aukcji (ManyToOne).
- Bid -> User: wiele ofert może zostać złożonych przez jednego użytkownika (ManyToOne).
- Auction -> currentHighestBid: aukcja przechowuje referencję do aktualnie najwyższej oferty (OneToOne).

W projekcie zdefiniowano także dwa typy wyliczeniowe: Category oraz AuctionStatus. Category określa kategorię przedmiotu, natomiast AuctionStatus przechowuje stan aukcji, np. ACTIVE lub CLOSED.

4.1. Diagram klas / ERD



5. Warstwa REST API

Warstwa REST API została zaimplementowana w kontrolerach oznaczonych adnotacją @RestController. W aktualnym repozytorium endpointy posiadają prefiks /api.

5.1. Endpointy użytkowników

Metoda	Endpoint	Opis	Kod sukcesu
POST	/api/users	Utworzenie nowego użytkownika.	201 Created
PUT	/api/users/{id}	Aktualizacja danych użytkownika.	200 OK
DELETE	/api/users/{id}	Usunięcie użytkownika.	204 No Content
GET	/api/users/{id}	Pobranie użytkownika po identyfikatorze.	200 OK
GET	/api/users	Pobranie listy	200 OK

		wszystkich użytkowników.	
--	--	--------------------------	--

5.2. Endpointy aukcji

Metoda	Endpoint	Opis	Kod sukcesu
POST	/api/auctions	Utworzenie nowej aukcji.	201 Created
PUT	/api/auctions/{id}	Aktualizacja danych aukcji.	200 OK
DELETE	/api/auctions/{id}	Logiczne zamknięcie aukcji poprzez ustawienie statusu CLOSED.	204 No Content
GET	/api/auctions/{id}	Pobranie aukcji po identyfikatorze.	200 OK
GET	/api/auctions?category={category}&status={status}	Pobranie listy aukcji z opcjonalnym filtrowaniem.	200 OK

5.3. Endpointy ofert

Metoda	Endpoint	Opis	Kod sukcesu
POST	/api/bids	Złożenie oferty na wskazaną aukcję przez wskazanego użytkownika.	201 Created
GET	/api/bids/{id}	Pobranie oferty po identyfikatorze.	200 OK
GET	/api/bids?auctionId={id}	Pobranie ofert dla konkretnej aukcji.	200 OK
GET	/api/bids?userId={id}	Pobranie ofert złożonych przez wskazanego użytkownika.	200 OK
GET	/api/bids	Pobranie listy wszystkich ofert.	200 OK

5.4. DTO używane przez API

DTO	Pola	Zastosowanie
UserRequestDto	username, password, email, firstName, lastName	Dane wejściowe przy tworzeniu i edycji użytkownika.
AuctionRequestDto	title, description, startingPrice, startDate, endDate, category	Dane wejściowe aukcji.
BidRequestDto	auctionId, userId, amount	Dane wejściowe oferty.

5.5. Przykładowe żądania JSON

Przykład utworzenia użytkownika:

```
{
  "username": "jan123",
  "password": "tajneHaslo",
  "email": "jan@example.com",
  "firstName": "Jan",
```

```
"lastName": "Kowalski"
}
```

Przykład utworzenia aukcji:

```
{
  "title": "Laptop Lenovo",
  "description": "Używany laptop w dobrym stanie",
  "startingPrice": 1500.00,
  "startDate": "2026-06-18T10:00",
  "endDate": "2026-06-20T18:00",
  "category": "ELECTRONICS"
}
```

Przykład złożenia oferty:

```
{
  "auctionId": 1,
  "userId": 2,
  "amount": 1700.00
}
```

6. Interfejs użytkownika

Interfejs użytkownika został zrealizowany przy użyciu Thymeleaf i pojedynczego widoku index.html. Strona główna została podzielona na zakładki oraz panel prezentujący najwyższe oferty.

- Rejestracja użytkownika – formularz tworzenia nowego konta i lista aktualnych użytkowników.
- Dodaj aukcję – formularz tworzenia nowej aukcji.
- Licytuj aukcje – lista aukcji wraz z formularzami składania ofert.
- Najwyższe oferty – stały panel boczny z podglądem aktualnych najwyższych ofert.

7. Walidacja danych

Projekt wykorzystuje walidację Bean Validation przez adnotacje takie jak @NotBlank, @NotNull, @Positive oraz @Email. Dane wejściowe są walidowane zarówno w warstwie REST API, jak i w formularzach widoku.

Obszar	Przykładowa reguła
Użytkownik	Login, hasło i email nie mogą być puste; email musi mieć poprawny format.
Aukcja	Tytuł, opis, cena, daty i kategoria są wymagane; cena musi być dodatnia.
Oferta	Kwota oferty musi być dodatnia; aukcja i użytkownik muszą istnieć.
Logika biznesowa	Oferta musi być wyższa od aktualnie najwyższej; licytacja jest możliwa tylko dla aktywnej aukcji.

8. Trwałość danych i konfiguracja

Aplikacja korzysta z trwałego przechowywania danych przez JPA/Hibernate i bazę SQLite. W pliku `application.properties` ustawiono połączenie JDBC do pliku `auction.db`, sterownik `org.sqlite.JDBC` oraz dialekt Hibernate dla SQLite.

Właściwość	Wartość / znaczenie
<code>spring.datasource.url</code>	<code>jdbc:sqlite:auction.db</code>
<code>spring.datasource.driver-class-name</code>	<code>org.sqlite.JDBC</code>
<code>spring.jpa.database-platform</code>	<code>org.hibernate.community.dialect.SQLiteDialect</code>
<code>spring.jpa.hibernate.ddl-auto</code>	<code>update</code>
<code>server.port</code>	<code>8080</code>

Konfiguracja `ddl-auto=update` powoduje automatyczne tworzenie i aktualizację schematu bazy na podstawie encji. Dane są przechowywane w pliku `auction.db` w katalogu uruchomienia aplikacji.

9. Instrukcja uruchomienia

1. Sklonować repozytorium z GitHub.
2. Otworzyć projekt w IntelliJ IDEA lub innym środowisku obsługującym Maven i Java 17.
3. Upewnić się, że zależności Maven zostały pobrane poprawnie.
4. Uruchomić klasę `com.psa.AuctionApplication`.
5. Po starcie aplikacji otworzyć przeglądarkę pod adresem `http://localhost:8080`.
6. Korzystać z interfejsu HTML albo testować REST API przy użyciu Swagger UI / Postman.

Alternatywnie projekt można uruchomić z poziomu terminala poleceniem Maven, np. `mvn spring-boot:run`.